Appendix A Setting up your development environment

This appendix covers

- Setting up Java
- Setting up Docker
- Setting up Kubernetes
- Setting up other tools

In this appendix, you'll find instructions for setting up your development environment and installing the tools we'll use throughout the book to build, manage, and deploy cloud native applications.

A.1 Java

All the examples in this book are based on Java 17, the latest long-term release of Java at the time of writing. You can install any of the OpenJDK 17 distributions. I'll be using Eclipse Temurin from the Adoptium project (**https://adoptium.net**), previously known as AdoptOpenJDK, but feel free to choose another one.

Managing different Java versions and distributions on your machine might be painful. I recommend using a tool like sdkman (**https://sdkman.io**) to install, update, and switch between different JDKs easily. On macOS and Linux, you can install sdkman as follows:

```
$ curl -s "https://get.sdkman.io" | bash
```

Refer to the official documentation for installation instructions for Windows.

Once it's installed, check all the available OpenJDK distributions and versions by running the following command:

```
$ sdk list java
```

Then choose a distribution and install it. For example, I can install the latest 17 version of Eclipse Temurin available at the moment of writing as follows:

```
$ sdk install java 17.0.3-tem
```

By the time you read this section, newer versions might be available, so please check the list returned from the `list` command to identify the latest one.

At the end of the installation procedure, sdkman will ask whether you want to make that distribution the default one. I recommend you say `yes` to ensure that you have access to Java 17 from all the projects you build throughout the book. You can always change the default version with the following command:

```
$ sdk default java 17.0.3-tem
```

Let's now verify the OpenJDK installation:

```
$ java --version
openjdk 17.0.3 2022-04-19
OpenJDK Runtime Environment Temurin-17.0.3+7 (build 17.0.3+7)
OpenJDK 64-Bit Server VM Temurin-17.0.3+7 (build 17.0.3+7, mixed mode)
```

You also have the option to change the Java version only within the context of the current shell:

```
$ sdk use java 17.0.3-tem
```

Finally, if you want to check which version of Java is configured in the current shell, you can do that as follows:

```
$ sdk current java
Using java version 17.0.3-tem
```

A.2 Docker

The Open Container Initiative (OCI), a Linux Foundation project, defines industry standards for working with containers (**https://opencontainers.org**). In particular, the OCI

Image Specification defines how to build container images, the OCI Runtime Specification defines how to run those container images, and the OCI Distribution Specification defines how to distribute them. The tool we use throughout the book to work with containers is Docker, which is compliant with the OCI specifications.

On the Docker website (**www.docker.com**), you can find instructions for setting up Docker in your local environment. I'll be using the latest versions available at the time of writing: Docker 20.10 and Docker Desktop 4.11.

- On Linux, you can install the Docker open source platform directly. It's also known as Docker Community Edition (Docker CE).
- On macOS and Windows, you have the option to use Docker Desktop, a commercial product built on top of Docker that makes it possible to run Linux containers from those operating systems. At the time of writing, Docker Desktop is free for personal use, education, non-commercial open source projects, and small businesses. Please read the Docker Subscription Service Agreement carefully before installing the software, and make sure you are compliant with it (**www.docker.com/legal**).

Docker Desktop provides support for both ARM64 and AMD64 architectures, meaning that you can run all the examples in this book on the new Apple computers with Apple Silicon processors.

If you work on Windows, Docker Desktop provides two types of setup: Hyper-V or WSL2. I recommend you choose the latter, since it offers better performance, and it's more stable.

Docker comes preconfigured to download OCI images from Docker Hub, a container registry hosting images for many popular open source projects, like Ubuntu, PostgreSQL, and Redis. It's free to use, but it's subjected to strict rate-limiting policies if you use it anonymously. Therefore, I recommend you create a free account on the Docker website (**www.docker.com**).

After creating an account, open a Terminal window and authenticate with Docker Hub (make sure your Docker Engine is running). Since it's the default container registry, you don't need to specify its URL:

```
$ docker login
```

When asked, insert your username and password.

Using the Docker CLI, you can now interact with Docker Hub to download images (*pull*) or upload your own (*push*). For example, try pulling the official Ubuntu image from Docker Hub:

```
$ docker pull ubuntu:22.04
```

Throughout the book, you'll learn more about using Docker. Until then, if you would like to experiment with containers, I'll leave you a list of useful commands for controlling the container life cycle (table A.1).

Table A.1 Useful Docker CLI commands for managing images and containers

Docker CLI command	What it does
docker images	Shows all images
docker ps	Shows the running containers
docker ps -a	Shows all containers created, started, and stopped
docker run <image>	Runs a container from the given image
docker start <name>	Starts an existing container
docker stop <name>	Stops a running container
docker logs <name>	Shows the logs from a given container
docker rm <name>	Removes a stopped container
docker rmi <image>	Removes an image

All the containers we build throughout the book are OCI-compliant and will work with any other OCI container runtime, such as Podman (**https://podman.io**). Should you decide to use a platform other than Docker, be aware that some tools we use for local development and integration testing might require additional configuration to work correctly.

A.3 Kubernetes

There are a few ways to install Kubernetes in your local environment. These are some of the most commonly used options:

- *minikube* (**https://minikube.sigs.k8s.io**) lets you run a local Kubernetes cluster on any operating system. It's maintained by the Kubernetes community.
- *kind* (**https://kind.sigs.k8s.io**) lets you run local Kubernetes clusters as Docker containers. It was primarily developed to test Kubernetes itself, but you can also use it for local development with Kubernetes. It's maintained by the Kubernetes community.
- *k3d* (**https://k3d.io**) lets you run local Kubernetes clusters based on k3s, a minimal distribution of Kubernetes implemented by Rancher Labs. It's maintained by the Rancher community.

Feel free to choose the tool that best fits your needs. I'll be using minikube throughout the book because of its stability and compatibility with all operating systems and architectures, including the new Apple Silicon computers. You should have at least two CPUs and 4 GB of free memory to use minikube to run all the examples in the book.

You can find the installation guide on the project's website (**https://minikube.sigs.k8s.io**). I'll be using the latest versions available at the time of writing: Kubernetes 1.24 and minikube 1.26. On macOS you can install minikube with Homebrew as follows:

```
$ brew install minikube
```

Running a local Kubernetes cluster with minikube requires a container runtime or a virtual machine manager. Since we are already using Docker, that's what we're going to use. Under the hood, any minikube cluster will run as a Docker container.

After installing minikube, you can start a new local Kubernetes cluster using the Docker driver. The first time you run this command, it will take a few minutes to download all the components needed to run the cluster:

```
$ minikube start --driver=docker
```

I recommend making Docker the default driver for minikube by running the following command:

```
$ minikube config set driver docker
```

To interact with the newly created Kubernetes cluster, you need to install kubectl, the Kubernetes CLI. Installation instructions are available on the official website (**https://kubernetes.io/docs/tasks/tools**). On macOS and Linux, you can install it with Homebrew as follows:

```
$ brew install kubectl
```

Then you can verify that the minikube cluster is started correctly and check that a node is running in your local cluster:

```
$ kubectl get nodes
NAME       STATUS   ROLES                  AGE    VERSION
minikube   Ready    control-plane,master   2m20s   v1.24.3
```

I recommend stopping minikube whenever you don't need it to free up resources in your local environment:

```
$ minikube stop
```

Throughout the book you'll learn more about using Kubernetes and minikube. Until then, if you would like to experiment with Kubernetes resources, I'll leave you with some useful commands (table A.2).

Table A.2 Useful Kubernetes CLI commands for managing Pods, Deployments, and Services

Kubernetes CLI command	What it does
kubectl get deployment	Shows all Deployments
kubectl get pod	Shows all Pods
kubectl get svc	Shows all Services
kubectl logs <pod_id>	Shows the logs for the given Pod
kubectl delete deployment <name>	Deletes the given Deployment
kubectl delete pod <name>	Deletes the given Pod
kubectl delete svc <service>	Deletes the given Service
kubectl port-forward svc <service> <host-port>:<cluster-port>	Forwards traffic from your local machine to within the cluster

A.4 Other tools

This section will present a series of helpful tools used throughout the book for performing specific tasks, such as security vulnerability scanning or HTTP interactions.

.4.1 HTTPie

HTTPie is a convenient "command-line HTTP and API testing client" (**https://httpie.org**). It's designed for humans and offers a superior user experience. Refer to the official documentation for installation instructions and more information on the tool.

On macOS and Linux, you can install it with Homebrew as follows:

```
$ brew install httpie
```

As part of the installation, you'll get two tools you can use from your Terminal window: `http` and `https`. For example, you can send a `GET` request as follows:

```
$ http pie.dev/get
```

4.2 Grype

In the context of supply chain security, we use Grype to scan Java codebases and container images for vulnerabilities (**https://github.com/anchore/grype**). The scanning happens locally on the machine where you run it, meaning that none of your files or artifacts are sent to an external service. That makes it a good fit for more regulated environments or air-gapped scenarios. Refer to the official documentation for more information.

On macOS and Linux, you can install it with Homebrew as follows:

```
$ brew tap anchore/grype
$ brew install grype
```

The tool is not available for Windows yet. If you are a Windows user, I recommend taking advantage of the Windows Subsystem for Linux 2 (WSL2) and installing Grype there. For more information on WSL2, you can refer to the official documentation (**https://docs.microsoft.com/en-us/windows/wsl/**).

4.3 Tilt

Tilt (**https://tilt.dev**) aims at providing a good developer experience when working on Kubernetes. It's an open source tool that offers features for building, deploying, and managing containerized workloads in your local environment. Refer to the official documentation for installation instructions (**https://docs.tilt.dev/install.html**).

On macOS and Linux, you can install it with Homebrew as follows:

```
$ brew install tilt-dev/tap/tilt
```

4.4 Octant

Octant (**https://octant.dev**) is an "open source developer-centric web interface for Kubernetes that lets you inspect a Kubernetes cluster and its applications." Refer to the official documentation for installation instructions (**https://reference.octant.dev**).

On macOS and Linux, you can install it with Homebrew as follows:

```
$ brew install octant
```

4.5 Kubeval

Kubeval (**www.kubeval.com**) is a convenient tool when you need to "validate one or more Kubernetes configuration files." We'll use it in a deployment pipeline to ensure that all our Kubernetes manifests are properly formatted and compliant with the Kubernetes API. Refer to the official documentation for installation instructions (**www.kubeval.com/installation/**).

On macOS and Linux, you can install it with Homebrew as follows:

```
$ brew tap instrumenta/instrumenta
$ brew install kubeval
```

4.6 Knative CLI

Knative is a "Kubernetes-based platform to deploy and manage modern serverless workloads" (**https://knative.dev**). The project provides a convenient CLI tool you can use to interact with Knative resources in a Kubernetes cluster. Refer to the official documentation for installation instructions (**https://knative.dev/docs/install/quickstart-install**).

On macOS and Linux, you can install it with Homebrew as follows:

```
$ brew install kn
```

Previous chapter

< 16 Serverless, GraalVM, and Knative

Next chapter

Appendix B Kubernetes in production with DigitalOcean >